SOFTWARE SECURITY TESTING

Building and Testing a Student Records Manager with Fuzzing and Static Analysis

Item	Details
Part 1 - Builder	Wrote **student_app.py**, a command-line program that stores students and grades, with bugs and security weaknesses planted on purpose.
Part 2 - Tester	Attacked the program with a **random fuzzer** (fuzz_task_1.py) and with the **static analysers Bandit and Pylint**.
Part 3 - Fixes	Fixed every finding (student_app_fixed.py) and re-ran all three tools.
Environment	Kali Linux (VirtualBox), Python 3, Bandit 1.9, Pylint 4

Abstract. One team member built a small Python application and a second member tested it without changing it, using two complementary techniques. Fuzzing ran the code 20,000 times with random input and found **8 unique crashes / wrong results** in the core logic. Static analysis read the code without running it: Bandit reported **9 security issues** (2 High, 4 Medium, 3 Low) and Pylint scored the code **8.45/10** with 12 messages. After the fixes, the fuzzer finds **0** problems, Bandit reports **0** issues and Pylint gives **10.00/10**. The report ends with a comparison of the two techniques and a before/after summary.

1. Part 1 - Building the Program

The program under test is a **Student Records Manager**. A record is typed as `name,age,grade1;grade2;grade3` (for example `Ali,20,90;85;77`). The program parses it, computes the average, converts it to a letter grade and stores the student in an SQLite database. It also offers a search, a calculator and an admin-only backup. The code is split into two areas so that each testing technique has a clear target:

- **Core logic** (parse_record, average, letter_grade, make_username, process_line) - pure functions with *input-handling bugs*, the target of the fuzzer.
- **Storage / admin features** - code with *security weaknesses* (SQL injection, eval, shell commands, pickle, MD5, hard-coded password), the target of static analysis.

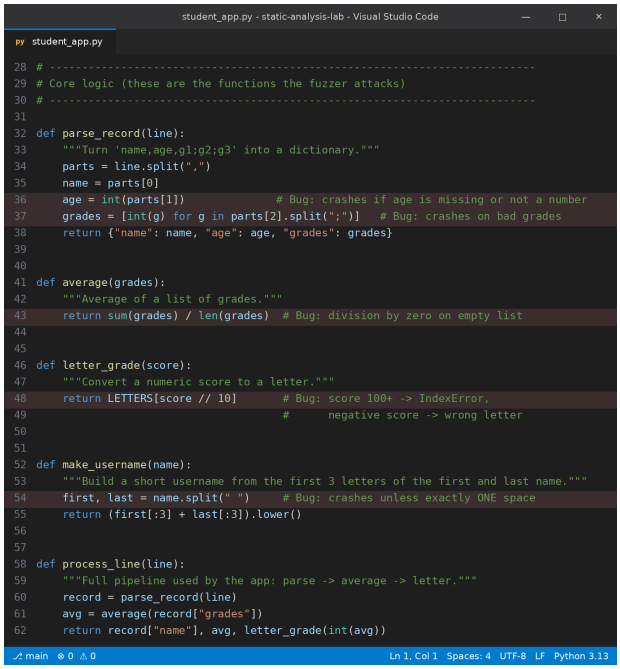


Figure 1 - Core logic of student_app.py (planted bugs highlighted).

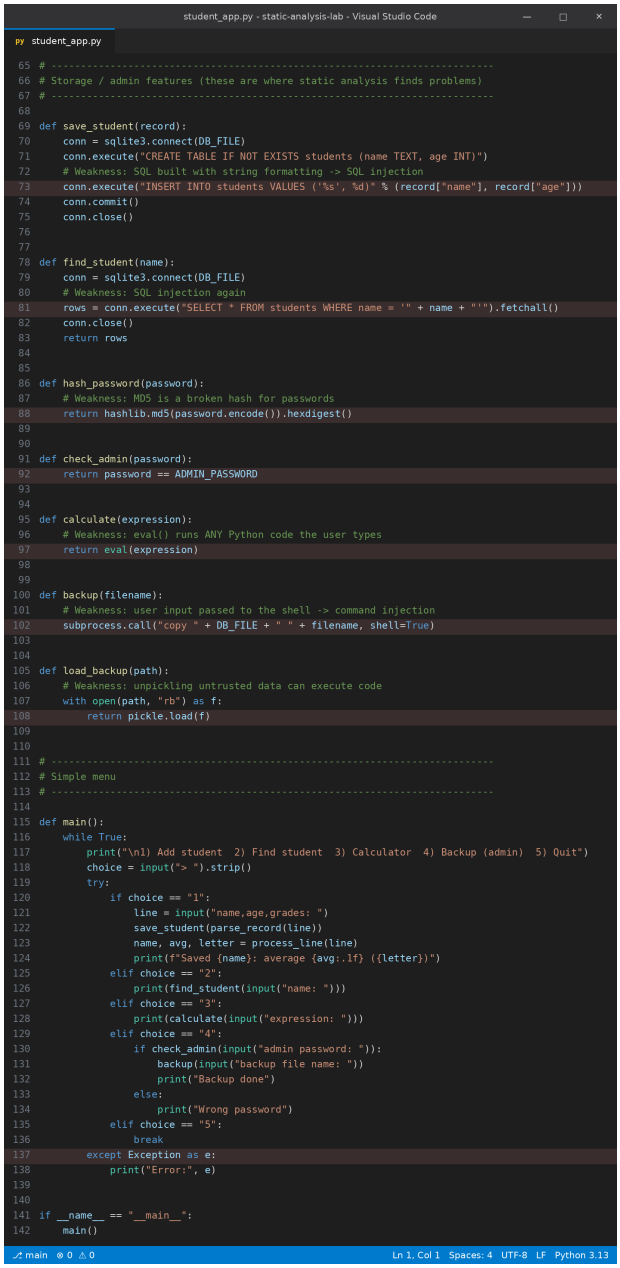


Figure 2 - Storage, admin features and menu (weaknesses highlighted).

Besides these, line 20 contains the hard-coded password `ADMIN_PASSWORD = "admin123"`, and lines 15 and 17 import the risky `pickle` and `subprocess` modules.

2. Part 2 - Testing the Program

2.1 Fuzz Testing

The tester wrote a **random (black-box) fuzzer**. Each of 20,000 iterations generates (a) fully random text and (b) *near-valid* records built from a list of tricky values (empty fields, negative ages, 999999, grades like 150 or 'x'), so the input passes the first checks and reaches deeper code. Every core function is called inside `try/except`; the first input that raises each (function, exception) pair is saved. A seed makes the run repeatable, and an extra **oracle** checks a bug that does not crash: a negative score must give 'F'.



Figure 3 - The fuzzer, fuzz_task_1.py (comments translated to English).

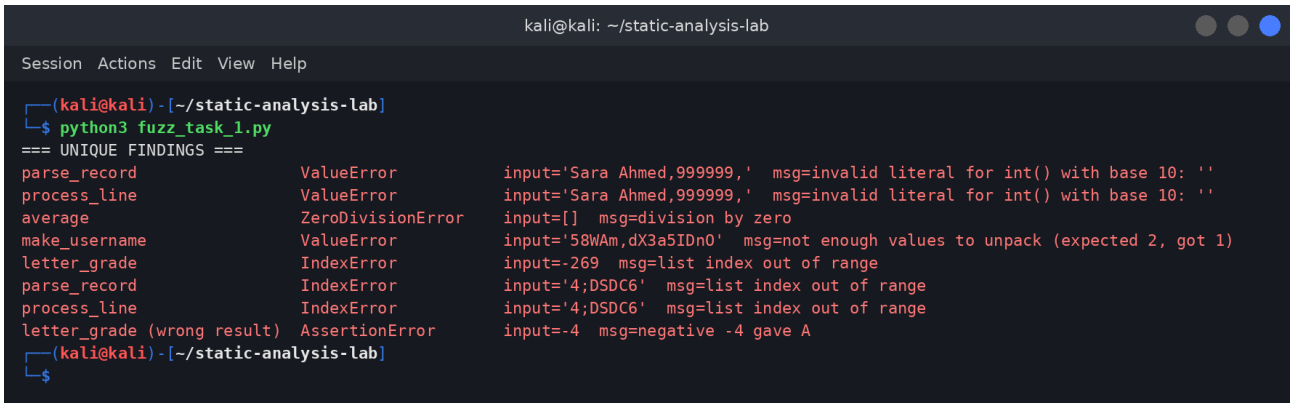


Figure 4 - Fuzzer output: 8 unique findings.

#	Function	Exception	Input that triggered it	Root cause
1-2	parse_record / process_line	ValueError	'Sara Ahmed,999999,'	int('') - the grade list is empty
3	average	ZeroDivisionError	[]	len(grades) is 0
4	make_username	ValueError	'58WAm,dX3a5IDnO'	name has no space, unpack fails

#	Function	Exception	Input that triggered it	Root cause
5	letter_grade	IndexError	-269	-269 // 10 = -27 is outside the list
6-7	parse_record / process_line	IndexError	'4;DSDC6'	fewer than 3 comma fields
8	letter_grade (oracle)	Wrong result	-4	-4 // 10 = -1 gives LETTERS[-1] = 'A'

Finding 8 is the most interesting: the program does not crash, it silently gives a failing student an **A**. Only the oracle check could catch it. Note also that the bug at score 100 (`100 // 10 = 10`, IndexError) was not hit because the fuzzer picks integers and the average of near-valid grades rarely lands at 100+; it is fixed anyway.

2.2 Static Analysis

Static analysis inspects the source code **without running it**. Two tools were used on Kali Linux: **Bandit**, a security linter that matches the code against known dangerous patterns (CWE), and **Pylint**, a quality linter. Commands used: `bandit -f txt -o bandit_report.txt student_manager.py`, `bandit -f json -o bandit_report.json student_manager.py` and `pylint student_manager.py > pylint_report.txt`.

Line	Bandit ID	Severity	Problem	CWE
102	B602	**High**	subprocess with shell=True - command injection	78
88	B324	**High**	MD5 used to hash passwords	327
73	B608	Medium	SQL built with % formatting - SQL injection	89
81	B608	Medium	SQL built with + concatenation - SQL injection	89
97	B307	Medium	eval() on user input - arbitrary code execution	78
108	B301	Medium	pickle.load on untrusted file - code execution	502
20	B105	Low	Hard-coded password 'admin123'	259
15	B403	Low	import pickle	502
17	B404	Low	import subprocess	78

Pylint reported 12 messages and a score of **8.45/10**:

Code	Count	Message	Lines
C0116	9	missing-function-docstring	69, 78, 86, 91, 95, 100, 105, 115
C0209	1	consider-using-f-string	73 (the same line as the SQL injection)
W0123	1	eval-used	97
W0718	1	broad-exception-caught (except Exception)	137

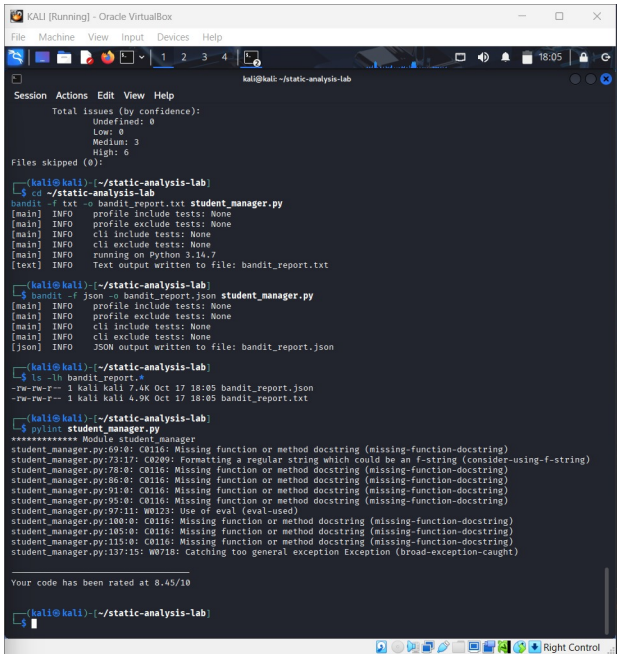


Figure 5 - Bandit reports written (confidence: Medium 3, High 6) and first Pylint run on Kali.

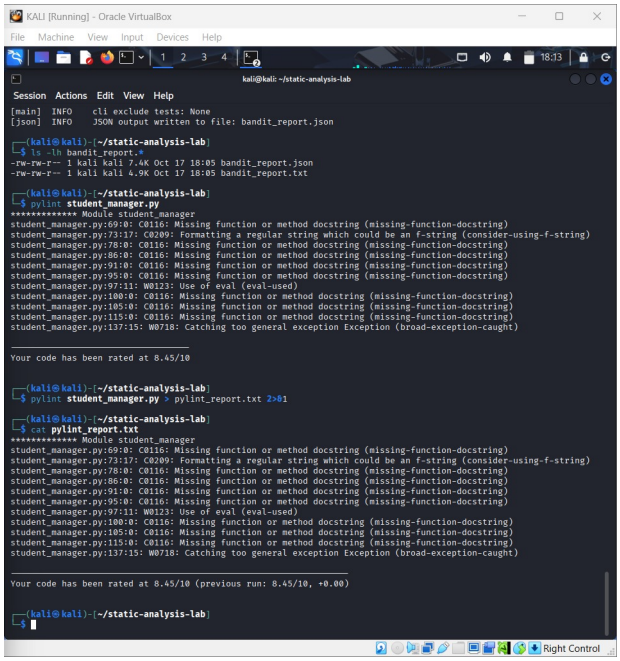


Figure 6 - Pylint output saved to pylint_report.txt (score 8.45/10).

3. Fixes

All findings were fixed in **student_app_fixed.py**. Bad user input is now rejected with a dedicated `InvalidRecordError` (a subclass of ValueError) that the menu catches and shows as a friendly message, instead of an unexpected crash.

Problem	Found by	Fix
Missing / non-numeric fields in parse_record	Fuzzer	Check exactly 3 fields, non-empty name, age 1-120, every grade 0-100
Division by zero in average	Fuzzer	Reject an empty grade list
IndexError and wrong 'A' in letter_grade	Fuzzer	Reject scores outside 0-100; min(score // 10, 9) so 100 = 'A'
make_username needs exactly one space	Fuzzer	name.split() on any whitespace, use first and last word
SQL injection (2 places)	Bandit B608	Parameterised queries with ? placeholders
Command injection via shell=True	Bandit B602/B404	shutil.copyfile, no shell; file name reduced to a safe base name
eval() calculator	Bandit B307, Pylint W0123	Small AST evaluator allowing only numbers and + - * /
pickle.load backup	Bandit B301/B403	Backups loaded as JSON (data only)
MD5 password hash	Bandit B324	Salted PBKDF2-HMAC-SHA256, 200,000 iterations
Hard-coded admin password	Bandit B105	Hash read from ADMIN_PASSWORD_HASH env var; hmac.compare_digest
Missing docstrings, f-string, broad except	Pylint	Docstrings added; specific exceptions caught
Student saved before validation	Code review	process_line validates before save_student

student_app_fixed.py - static-analysis-lab - Visual Studio Code

student_app_fixed.py

```
    """Turn 'name,age,g1;g2;g3' into a dictionary, validating every field."""
    parts = line.split(",")
    if len(parts) != 3:
        raise InvalidRecordError("expected exactly 3 fields: name,age,grades")
    name, age_text, grades_text = (p.strip() for p in parts)
    if not name:
        raise InvalidRecordError("name is empty")
    if not age_text.isdigit() or not 1 <= int(age_text) <= 120:
        raise InvalidRecordError("age must be a whole number between 1 and 120")
    grades = []
    for g in grades_text.split(";"):
        g = g.strip()
        if not g.isdigit() or int(g) > 100:
            raise InvalidRecordError(f"bad grade {g!r} (must be 0-100)")
        grades.append(int(g))
    return {"name": name, "age": int(age_text), "grades": grades}


def average(grades):
    """Average of a non-empty list of grades."""
    if not grades:
        raise InvalidRecordError("cannot average an empty list of grades")
    return sum(grades) / len(grades)


def letter_grade(score):
    """Convert a numeric score (0-100) to a letter."""
    if not 0 <= score <= 100:
        raise InvalidRecordError(f"score {score} is outside 0-100")
    return LETTERS[min(score // 10, 9)]          # 100 maps to 'A'


def make_username(name):
    """Build a username from the first 3 letters of the first and last name."""
    words = name.split()                          # any amount of whitespace
    if len(words) < 2:
        raise InvalidRecordError("name needs a first and a last name")
    return (words[0][:3] + words[-1][:3]).lower()


def process_line(line):
    """Full pipeline used by the app: parse -> average -> letter."""
    record = parse_record(line)
    avg = average(record["grades"])
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

Figure 7 - Fixed core logic: every input is validated.

student_app_fixed.py - static-analysis-lab - Visual Studio Code

student_app_fixed.py

```
def save_student(record):
    """Insert a student using a parameterised query (no SQL injection)."""
    with sqlite3.connect(DB_FILE) as conn:
        conn.execute("CREATE TABLE IF NOT EXISTS students (name TEXT, age INT)")
        conn.execute("INSERT INTO students VALUES (?, ?)",
                     (record["name"], record["age"]))


def find_student(name):
    """Look a student up by name using a parameterised query."""
    with sqlite3.connect(DB_FILE) as conn:
        return conn.execute("SELECT * FROM students WHERE name = ?",
                            (name,)).fetchall()


def hash_password(password, salt=None):
    """Salted PBKDF2-SHA256 instead of MD5. Returns 'salt$hash' in hex."""
    salt = salt or os.urandom(16)
    digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 200_000)
    return salt.hex() + "$" + digest.hex()


def check_admin(password):
    """Compare against the stored hash in constant time."""
    if "$" not in ADMIN_PASSWORD_HASH:
        return False
    salt_hex, _ = ADMIN_PASSWORD_HASH.split("$", 1)
    candidate = hash_password(password, bytes.fromhex(salt_hex))
    return hmac.compare_digest(candidate, ADMIN_PASSWORD_HASH)


_OPS = {ast.Add: operator.add, ast.Sub: operator.sub, ast.Mult: operator.mul,
        ast.Div: operator.truediv, ast.USub: operator.neg, ast.UAdd: operator.pos}


def calculate(expression):
    """Safe calculator: only numbers and + - * / are allowed (no eval)."""
    def walk(node):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            return node.value
        if isinstance(node, ast.BinOp) and type(node.op) in _OPS:
            return _OPS[type(node.op)](walk(node.left), walk(node.right))
        if isinstance(node, ast.UnaryOp) and type(node.op) in _OPS:
            return _OPS[type(node.op)](walk(node.operand))
        raise InvalidRecordError("only numbers and + - * / are allowed")
    if len(expression) > 100:
        raise InvalidRecordError("expression too long")
    return walk(ast.parse(expression, mode="eval").body)


def backup(filename):
    """Copy the database with shutil - no shell, and the name is sanitised."""
    safe_name = Path(filename).name               # strips any ../ or /path
    if not safe_name or not safe_name.replace(".", "").replace("_", "").isalnum():
        raise InvalidRecordError("backup name may only contain letters, digits, '.' and '_'")
    BACKUP_DIR.mkdir(exist_ok=True)
    shutil.copyfile(DB_FILE, BACKUP_DIR / safe_name)


def load_backup(path):
    """Load backup data as JSON (data only - cannot execute code like pickle)."""
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)
```

main ⊗ 0 ⚠ 0 Ln 1, Col 1 Spaces: 4 UTF-8 LF Python 3.13

Figure 8 - Fixed storage and admin features.

4. Re-testing and Comparison

The same fuzzer (same seed, same 20,000 iterations) was re-run against the fixed module; the only change is that the controlled `InvalidRecordError` is counted as correct behaviour, not as a bug. Bandit and Pylint were run again with the same settings.

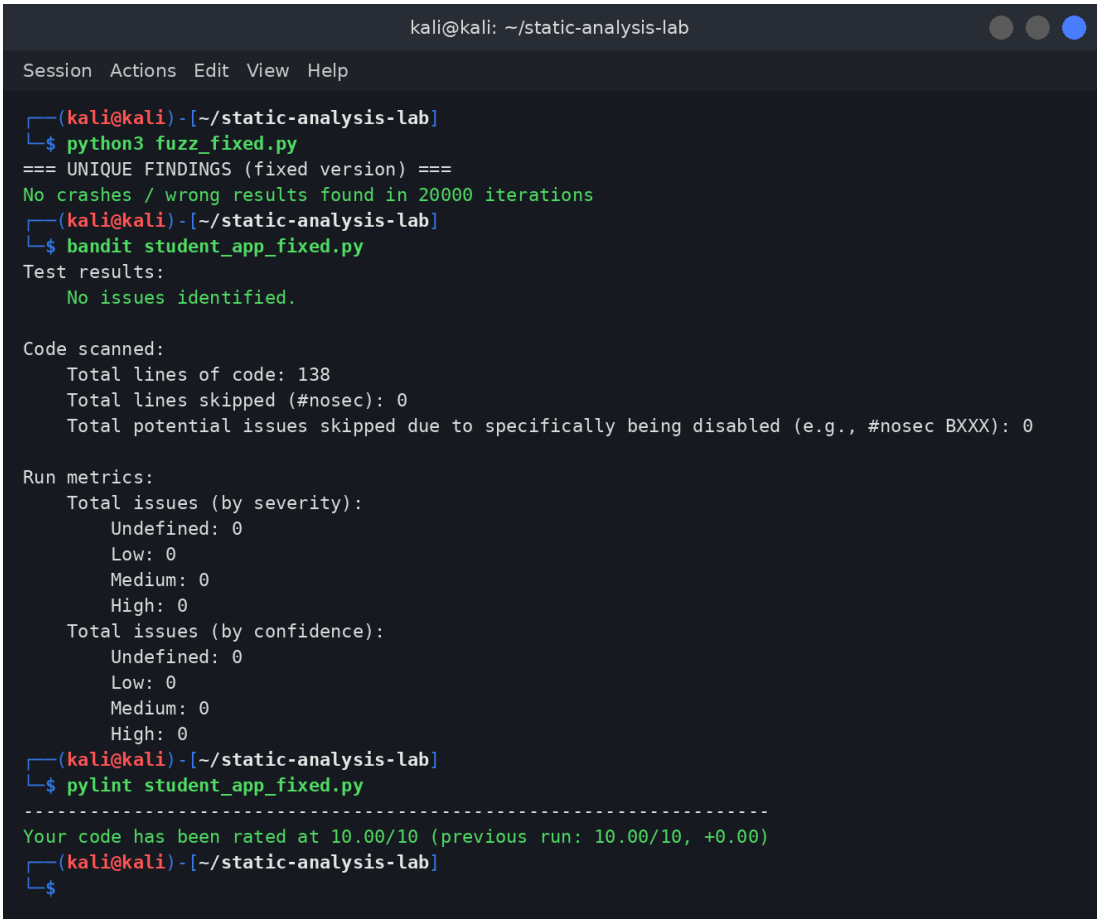


Figure 9 - Re-test of the fixed version.

4.1 Before vs After

Metric	Original	Fixed
Fuzzer unique findings	**8**	**0**
Bandit High / Medium / Low	**2 / 4 / 3**	**0 / 0 / 0**
Pylint score	8.45 / 10 (12 messages)	**10.00 / 10** (0 messages)
Bad input such as 'Ali,abc,'	Crash / generic error	Clear 'Invalid input' message
Calculator input __import__('os')	Executes code	Rejected

4.2 Fuzzing vs Static Analysis

Aspect	Fuzzing	Static analysis (Bandit + Pylint)
How it works	Runs the code with many random inputs	Reads the source code without running it
What it found here	8 runtime crashes and 1 logic error in the core logic	9 security issues + 12 quality messages in storage/admin code
Overlap	None - it never touched SQL, eval, shell or pickle	None - it saw no problem in parse_record, average, letter_grade
False positives	None: every finding came with a real input	Some: B403/B404 only flag an import, not a real flaw
Silent logic bugs	Yes, if an oracle is written (negative = 'A')	No - the code is valid Python
Coverage	Only code it reaches; may miss rare values (score 100)	Every line, every path, in seconds
Proof / reproduction	Gives the exact failing input	Gives file, line and CWE, but no exploit input
Cost	Must write a harness and run many iterations	One command, no setup

Conclusion. The two techniques found **completely different** problems - neither alone would have revealed all 17 issues. Fuzzing is best for robustness: it proves with a concrete input that the program crashes or returns wrong data. Static analysis is best for known dangerous patterns such as injection, unsafe deserialisation and weak cryptography, which random input rarely triggers. Using both, then fixing and re-testing, took the program from 8 crashes, 9 security issues and 8.45/10 to **0 crashes, 0 issues and 10/10**.

4.3 Lessons Learned

- **Validate input at the boundary.** Seven of the eight fuzzer findings came from trusting the format of user input; one validation function removed all of them.
- **Never build code, SQL or shell commands from strings.** Parameterised queries, an AST evaluator and shutil remove whole classes of injection instead of patching single cases.
- **Crash-free is not the same as correct.** The negative-score bug only appeared because the fuzzer had an oracle that knew the expected answer.
- **Some static warnings are informational.** B403/B404 flag imports; the real fix was to stop needing pickle and subprocess at all, so the imports disappeared.
- **Re-test after fixing.** Running the same seed and the same tools again is what proves the fixes work and did not break valid input (e.g. 'Ali,20,90;85;100' still gives A).